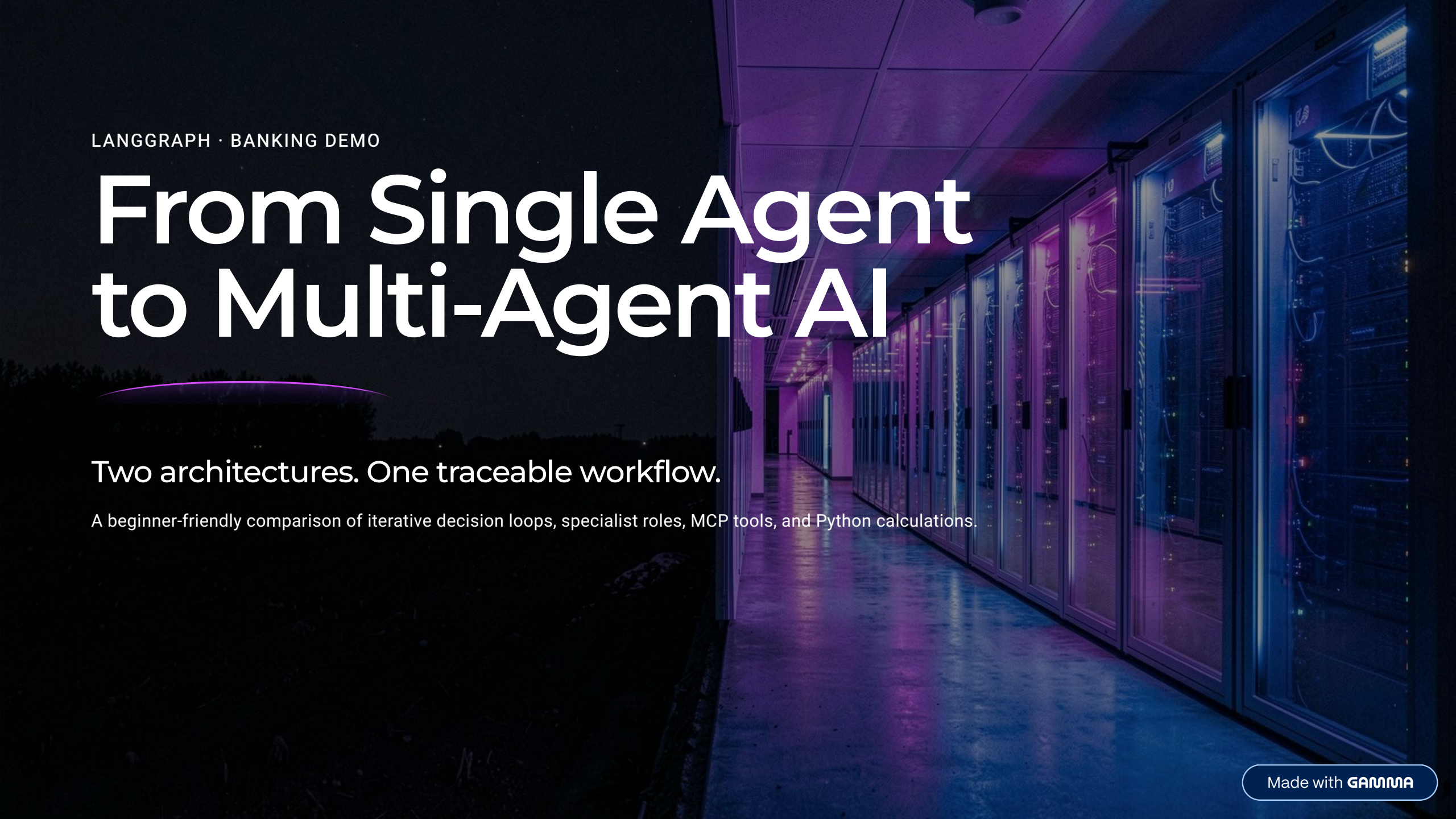


LANGGRAPH · BANKING DEMO

From Single Agent to Multi-Agent AI

Two architectures. One traceable workflow.

A beginner-friendly comparison of iterative decision loops, specialist roles, MCP tools, and Python calculations.

Made with GAMMA

01 · SYSTEM BOUNDARIES

The banking demo is a small system with clear boundaries

- Streamlit UI
- LangGraph Workflow
- Groq LLM
- MCP Client & Server

The data is mocked demo data, not production banking data.



One demo, five boundaries

Streamlit, Groq, LangGraph, MCP, and Python each have a clear job.

Made with GAMMA

02 · SINGLE-AGENT LOOP

One agent can reason, act, and loop

The LLM selects the next permitted step instead of writing the whole plan upfront.

decide

The LLM chooses the next action.

execute_action

Run the selected MCP tool or Python analysis.

finalize

Prepare the final answer.

START → decide → conditional routing → execute_action → decide again → finalize → END

Possible actions: get_account_balance, get_recent_transactions, analyze_transactions, and finish.



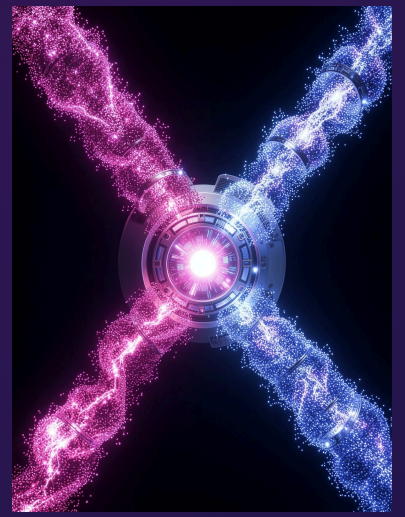
Made with GAMMA

03 · MULTI-AGENT ROLES

Multiple agents turn responsibilities into roles



All agents can use the configured Groq model; different responsibilities and prompts do not require separate models.



Different responsibilities and prompts do not require separate models.

Made with GAMMA

04 · ARCHITECTURE TRADE-OFF

Single-agent simplicity trades off against multi-agent coordination

Dimension	Single agent	Multi-agent
Decision-making	One LLM-driven loop	Supervisor-coordinated specialist roles
Workflow	One agent selects tools	Specialists handle distinct roles
Orchestration	Conditional routing	Supervisor-led routing
Shared state	One agent state	State coordinates specialist findings
Complexity	Simpler to build and debug	More coordination and failure modes

Use a single agent when tasks are compact and tool choices tightly related; use multiple agents when responsibilities can be isolated and composed.

Made with GAMMA

05 · TRACE THE REQUEST

A user question becomes a traceable workflow

Example path: “How much did I spend recently?”

01

Streamlit

Captures the natural-language banking question.

02

Shared state

LangGraph stores the request and current findings.

03

Supervisor

Routes work to transaction or budget analysis.

04

Specialists

MCP and Python return structured findings.

05

Response

The response agent writes one grounded answer.

Made with GAMMA

06 · SEPARATE THE JOBS

Keep deterministic work in Python and reasoning in the LLM



MCP tools

Retrieve account balances and transactions through MCP.



Python

Calculate totals, categories, averages, and other exact values.



LLM

Choose the next role, interpret findings, and explain results in plain language.

ILLUSTRATIVE STATE SHAPE

```
state = {
  "messages": ...,
  "next_agent": ...,
  "findings": ...,
  "final_response": ...
}
```

The graph makes each transition explicit and debuggable.

Made with GAMMA

07 · DESIGN RULE

Start with one loop, then split where the work becomes distinct

START

Single agent

Strong starting point for a focused tool-using workflow.

SPLIT WHEN

Work becomes distinct

Specialist responsibilities, routing, or evaluation boundaries are clear.

KEEP VISIBLE

Shared state

LangGraph makes either design inspectable through explicit transitions.

MCP keeps banking capabilities behind tool interfaces; Python protects calculation accuracy.

Multi-agent does not mean more models. It means clearer roles, routing, and shared state.

Made with GAMMA